Guía para la defensa

Guía para la defensa en clase

Orden sugerido, pensado para que no haya tiempos muertos.

Antes de empezar (10 minutos antes)

```
# 1. Docker Desktop abierto
# 2. Levantar Jenkins
export REPO_URL=https://github.com/USUARIO/catalogo-videojuegos.git
./scripts/levantar-jenkins.sh

# 3. Dejar corriendo un primer build para que la caché de Maven esté caliente
#    (el primer build baja todas las dependencias y tarda varios minutos)
#    Build Now desde http://localhost:8080/job/catalogo-videojuegos/

# 4. Verificar que producción responde
./scripts/estado.sh
```

Pestañas abiertas del navegador:

1. `http://localhost:8080/job/catalogo-videojuegos/` — Jenkins
2. `http://localhost:8091` — producción
3. El repositorio en GitHub
4. El `Jenkinsfile` en el editor

1. Arquitectura y estructura (≈3 min)

- Mostrar el árbol del proyecto: los dos módulos, `app-web`, el `Jenkinsfile`.
- Explicar la dependencia entre módulos: `app-web → clasificacion → catalogo`.
- Mostrar el diagrama de la sección 2.1 del informe: desarrollador → GitHub → Jenkins → contenedor de producción.

2. El pipeline (≈3 min)

Recorrer el `Jenkinsfile` de arriba hacia abajo, sin leer código línea por línea:

- `triggers`: webhook + polling cada 2 minutos.
- Las ocho etapas y, en particular, por qué las pruebas unitarias y las de integración están separadas (surefire vs. failsafe).
- El `post { failure }`: qué pasa con producción cuando algo falla.

3. La aplicación funcionando (≈2 min)

En `http://localhost:8091`: agregar un juego, filtrar por género, mostrar la cabecera que dice `PRODUCCIÓN` y el número de versión.

4. Caso 1 — Integración exitosa (≈4 min)

Cambio chico y visible. Por ejemplo, en `modulo-clasificacion/src/main/java/uy/edu/um/pa/clasificacion/Genero.java` agregar:

```
PLATAFORMAS("Plataformas"),
```

```
git add -A
git commit -m "Agrega el genero Plataformas"
git push
```

Mientras Jenkins corre, mostrar el tablero de etapas. Al terminar, recargar `http://localhost:8091`: el nuevo género aparece en el desplegable del formulario y los juegos cargados antes siguen estando.

5. Caso 2 — Fallo en las pruebas (≈4 min)

Romper a propósito la normalización de géneros. En `modulo-clasificacion/src/main/java/uy/edu/um/pa/clasificacion/ServicioClasificacion.java`, en el método `normalizar`, devolver el texto tal cual:

```
public String normalizar(String generoLibre) {
    return generoLibre;   // ← cambio que rompe las pruebas
}
```

```
git commit -am "Simplifica la normalizacion de generos"
git push
```

Qué mostrar:

1. El pipeline arranca solo y se detiene en **Pruebas unitarias** (rojo).
2. En *Test Result*: `ServicioClasificacionTest.normalizaGenero` esperaba `"RPG"` y recibió `"rpg"`.
3. **Las etapas de imagen y despliegue nunca se ejecutan.**
4. Recargar `http://localhost:8091`: producción sigue intacta con la versión anterior.
5. Explicar el procedimiento: se corrige o se revierte, se vuelve a pushear, y recién ahí el cambio llega a producción.

```
git revert --no-edit HEAD
git push
```

El pipeline vuelve a correr y queda todo en verde.

Preguntas que probablemente hagan

¿Por qué el pipeline está en el repositorio y no configurado en Jenkins? Porque así el proceso de construcción versiona junto con el código: si una rama necesita una etapa distinta, la trae en su propio `Jenkinsfile`, y cualquier cambio al pipeline queda en el historial de Git.

¿Cómo sabe Jenkins que hubo un cambio? Dos mecanismos: el webhook de GitHub (`githubPush`) y, como respaldo porque Jenkins corre en localhost y GitHub no lo alcanza, polling del SCM cada dos minutos.

¿Qué pasa si falla el despliegue y no las pruebas? El bloque `post { failure }` detecta que había una imagen anterior y hace rollback. Cada build etiqueta su imagen con el número de build, así que volver atrás es un `docker run`.

¿Por qué archivo JSON y no una base de datos? Porque la persistencia no es el objeto del entregable y una base agregaría un contenedor, credenciales y migraciones al ambiente de producción sin aportar nada al proceso de CI/CD. El archivo está en un volumen, que es lo que demuestra que los datos sobreviven al despliegue.

¿Dónde corre Maven? No se instala: el repositorio trae el Maven Wrapper, así que Jenkins y los desarrolladores usan exactamente la misma versión (3.9.9).